

STL (Standard Template Library)

C++

- STL is a collection of generic (template based) classes and functions.
- Provides ready to use data structures, algorithms and utility functions.
- Included in `<bits/stdc++.h>` or individual headers like `<vector>`, `<queue>`, `<stack>`, etc.

① `pow(a, b)` $\rightarrow a^b$

- defined in `<cmath>`
- returns double (use carefully for integer powers as it may have precision issues for large values).

② `pair<int, int>`

- stores 2 values (can be of different types)
- access using `p.first` $\rightarrow$ first value
`p.second` $\rightarrow$ second value
- example : `pair<int, int> p = { x, y };`

③ `vector` (dynamic array)

- contiguous memory (like array) but size can change
- `#include <vector>`
- `vector<int> v;`
- fast access ($O(1)$) using index.

③ vector (contd...)

- `v.push_back(x)` → add element at end
- `v.emplace_back(x)` → constructs element in place (slightly faster)
- `v.size()` → total elements (returns `size_t`)
- `v.begin()` → iterator to first element
- `v.end()` → iterator to one past last element
↳ (not the last element)
- `v.rbegin()` → reverse iterator to last element
- `v.rend()` → reverse iterator to before first element
- `v.front()` → first element
- `v.back()` → last element
- `v.pop_back()` → remove last element
- `v.at(i)` → element at index `i` (with bounds check)
- `v.insert(v.begin()+i, x)` → insert `x` at position `i`
- `v.erase(v.begin()+i)` → erase element at position `i`
- `v.clear()` → remove all elements (size becomes 0)
- `swap(v1, v2)` → swap contents of two vectors

Traversal

```
for (auto x : v) {  
    cout << x << " ";  
}
```

→ range based for loop (preferred)

④ List

- doubly Linked List (each element in separate memory)
- insertion / deletion from front is fast $O(1)$
- used when frequent insert / delete in middle / front is needed.
- `#include <List>`
- `List<int> L;`
- `L.push-front(x)`, `L.push-back(x)`, `L.insert()`, `L.erase()`

⑤ deque (double ended queue)

- supports insertion and deletion from both front and back
- Like a dynamic array of fixed size blocks (not a linked List)
- `#include <deque>`
- `deque<int> dq;`
- `dq.push-front(x)`, `dq.push-back(x)`

⑥ stack (LIFO)

- Last In First Out
- `#include <stack>`
- `stack<int> s;`
- `s.push(x)` → insert
- `s.top()` → return top element
- `s.pop()` → remove top element
- `s.empty()` → check if empty
- `s.size()` → number of elements

⑦ queue (FIFO)

- First In First Out
- `#include <queue>`
- `queue<int> q;`
- `q.push(x)` → insert at rear
- `q.front()` → first element
- `q.pop()` → remove first element
- `q.empty()`
- `q.size()`

⑧ priority_queue

- stores elements so that largest element is on top (by default)
- implemented using heap
- `#include <queue>`
- `priority_queue<int> pq;`
- `pq.push(x)` → insert element
- `pq.top()` → returns largest element
- `pq.pop()` → remove largest element

Min heap (smallest element on top)

- `priority_queue<int, vector<int>, greater<int>> pq;`

⑨ set

- stores unique elements in sorted (ascending) order
- `#include <set>`
- `set<int> s;`
- `s.insert(x)` → $O(\log n)$
- `s.find(x)` → returns iterator (or `s.end()` if not found)
- `s.count(x)` → 1 if present, else 0
- `s.erase(x)` → remove element
- `s.lower_bound(x)` → iterator to first element $\geq x$
- `s.upper_bound(x)` → iterator to first element $> x$

★ multiset → can store repeating elements.

10 map

- stores key-value pairs in sorted order by key
- keys are unique
- `#include <map>`
- `map<int, int> mp;`
- `mp[key] = value`
- `mp.find(x)` → returns iterator
- `mp.erase(x)` → removes key
- useful when order by key is needed.

11 unordered_map

- no sorted order of storing
- keys are unique
- average $O(1)$ insert / find / erase
- `#include <unordered_map>`
- `unordered_map<int, int> ump;`
- useful when ordering is not required and fast lookup is needed.

12 multimap

- stores key-value pairs in sorted order by key
- duplicate keys are allowed
- `#include <map>`
- `multimap<int, int> mm;`
- `mm.equal_range(x)` → returns pair of iterators covering all entries with key x.

13 Useful STL algorithms

- `next_permutation(v.begin(), v.end())`
- `prev_permutation(v.begin(), v.end())`
- `* max_element(v.begin(), v.end())`
- `* min_element(v.begin(), v.end())`
- `accumulate(v.begin(), v.end(), 0)`
- `reverse(v.begin(), v.end())`
- `comparator` → custom comparison function used in sort / priority logic.